

Slippery SOP

Edge Cases in the Same Origin Policy

Agenda

- **whoami**
- The Same-Origin Policy
- Cookies
- DNS Rebinding and Demo

whoami

- Collin Berman (collinberman1@gmail.com)
- Capital One Offensive Security
 - Pentester
 - Web, cloud, crypto
- Seattle, WA
 - Hiking, climbing, camping, skiing



What is this talk?

- Why doesn't the Same Origin Policy protect us from Cross-Site Request Forgery?
- Not a CORS talk
- Understanding the boundaries of the SOP
- Mostly a collection of others' research
- A reference: wide and shallow
- Bonus vuln at the end

Agenda

- whoami
- **The Same-Origin Policy**
- Cookies
- DNS Rebinding and Demo

The Same-Origin Policy

“The same-origin policy controls interactions between two different origins”

- [Mozilla Developer Network](#)

Interactions:

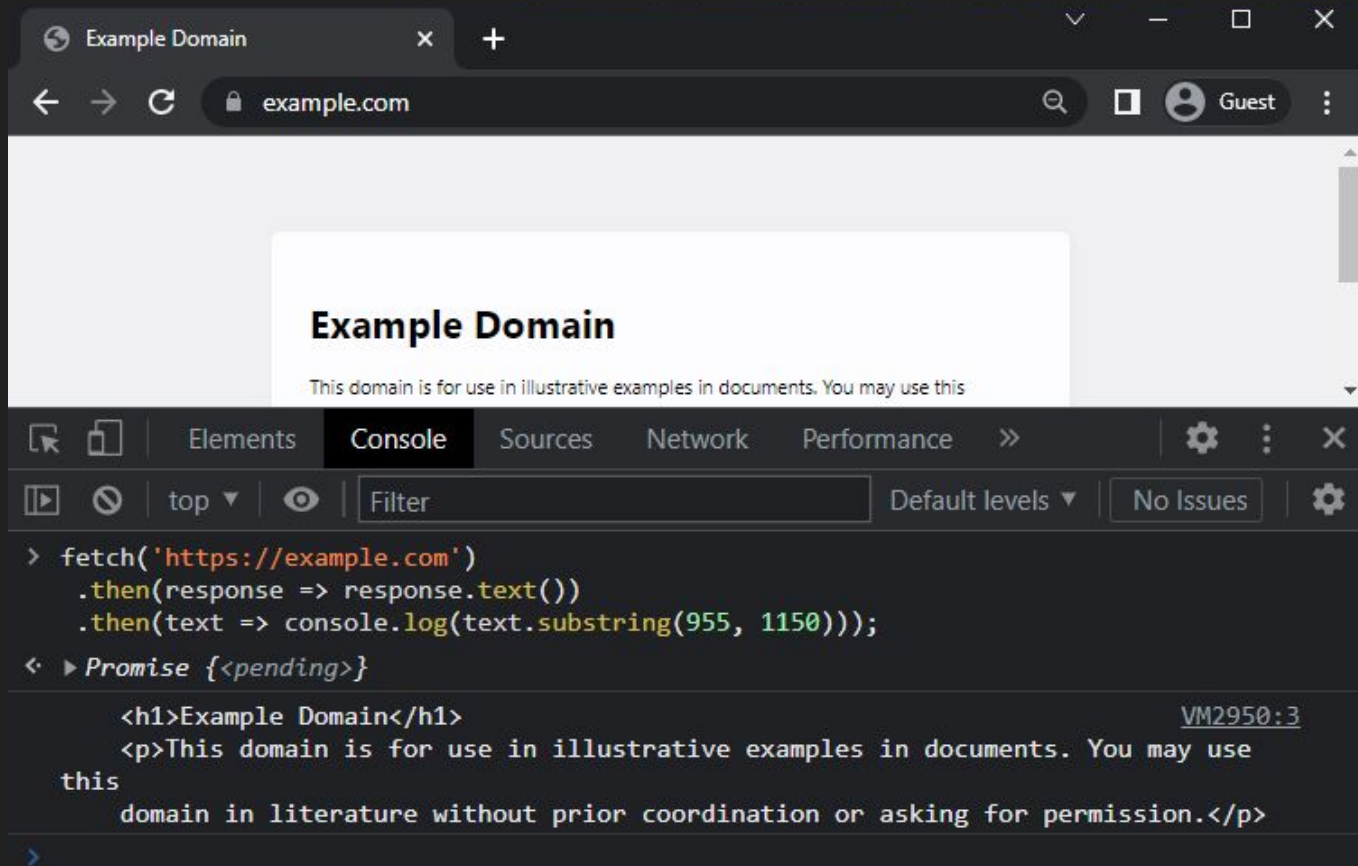
- Reading
- Writing
- Embedding content

Origins:

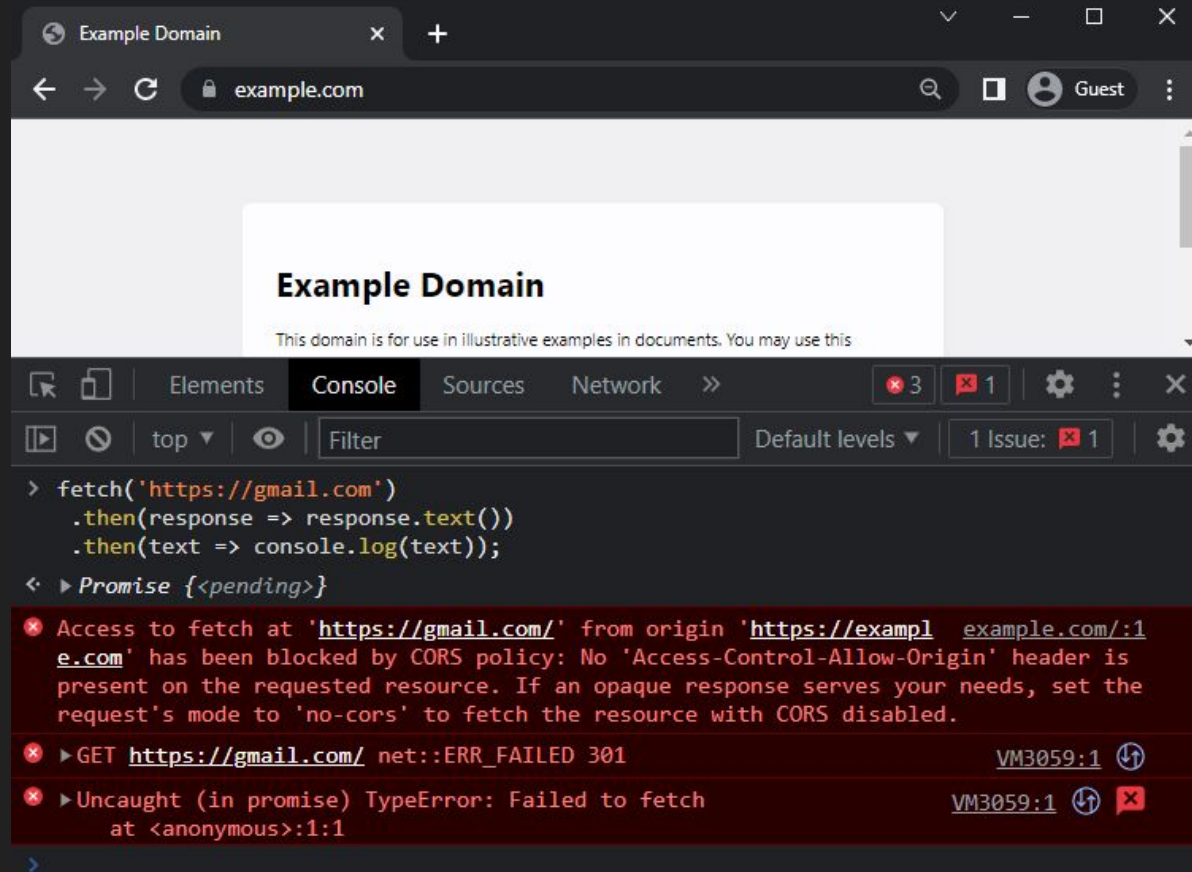
- Protocol
- Host
- Port

`https://example.com:443`

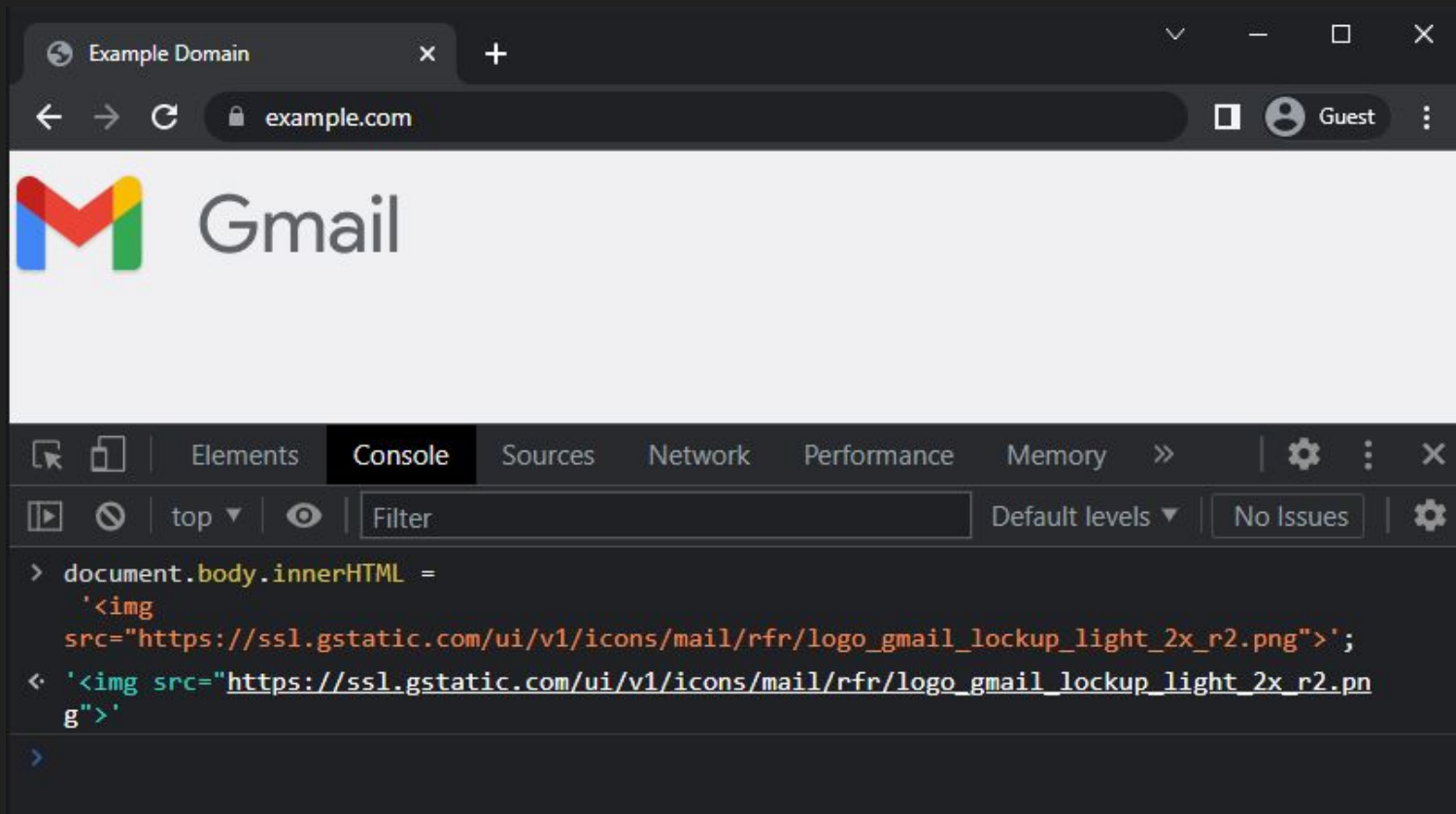
Same-Origin Reads



Cross-Origin Reads



Cross-Origin Embeds



Cross-Origin Writes AKA CSRF

- Clicking through links
- Redirects
- Form submissions
- XMLHttpRequest
 - / **fetch** / Background Fetch
 - / sendBeacon / <a ping>

IT'S A CROSS-SITE REQUEST, SIR



BUT IT CHECKS OUT

CSRF



Browse / view ad

"Make a POST request to
`https://example.com/change_email`

 POST
`https://example.com/change_email`
`email=attacker@evil.com`



`https://evil.com`



`https://example.com`

Same Origin Policy for fetch (XMLHttpRequest)

- Approximately, fetch is limited to what Form Submission can do
- Allowed methods: GET, POST, HEAD
- Only “safe” request headers are allowed:
 - Accept, Accept-Language, Content-Language
 - DPR, Dprn, Save-Data, Viewport-Width, Width
 - Content-Type (limited to a few options)
- Content-Type is restricted to:
 - application/x-www-form-urlencoded
 - multipart/form-data
 - text/plain
- **These Cross-Site limitations induce CSRF mitigations**

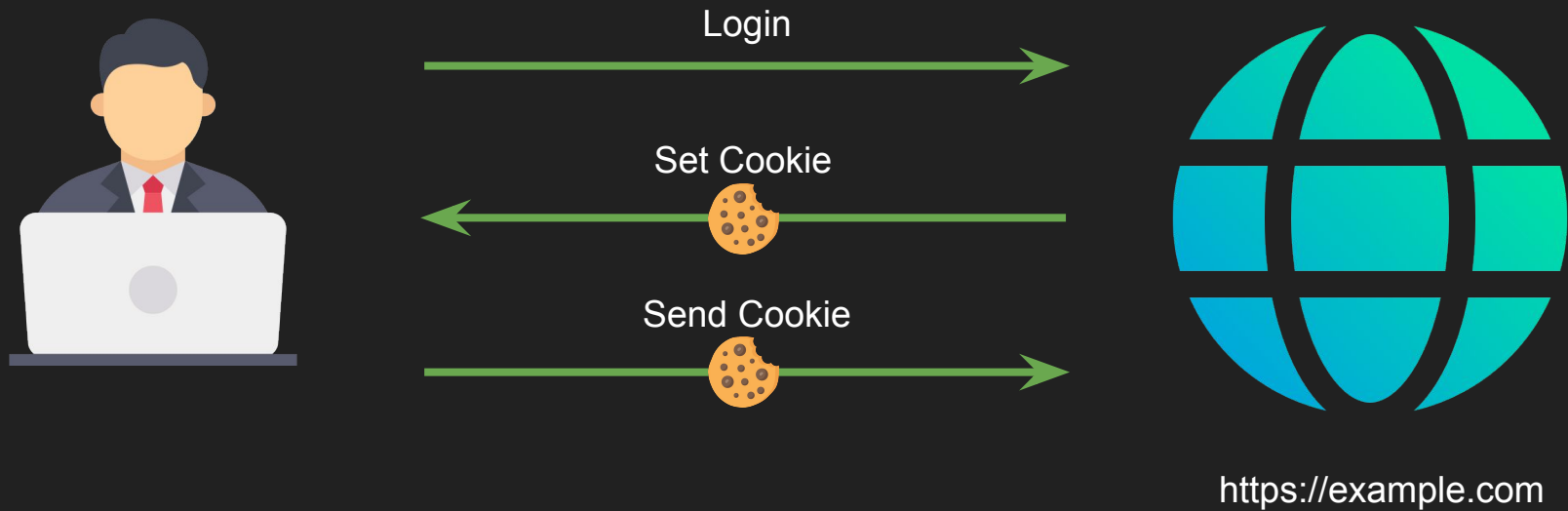
Weakening the SOP

- Obligatory CORS mention
- JavaScript: `document.domain = "parentdomain.com";`

Agenda

- whoami
- The Same-Origin Policy
- **Cookies**
- DNS Rebinding and Demo

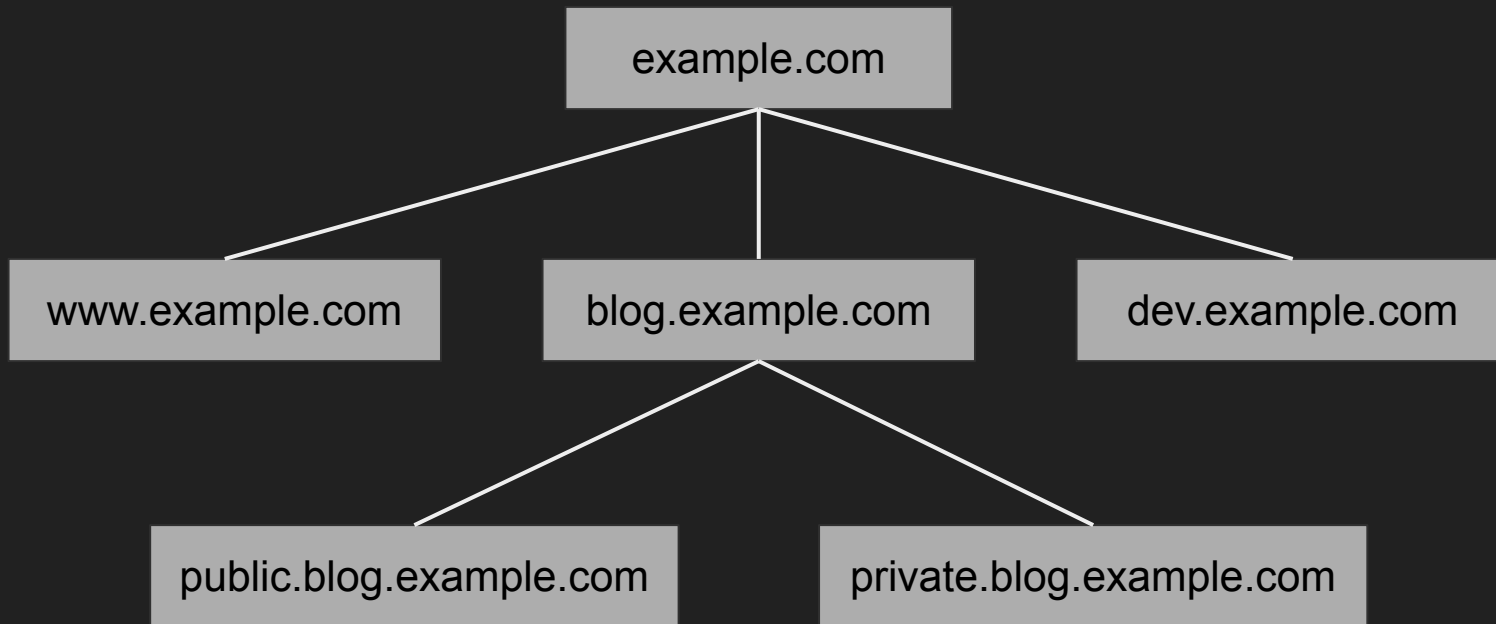
Cookie Flow



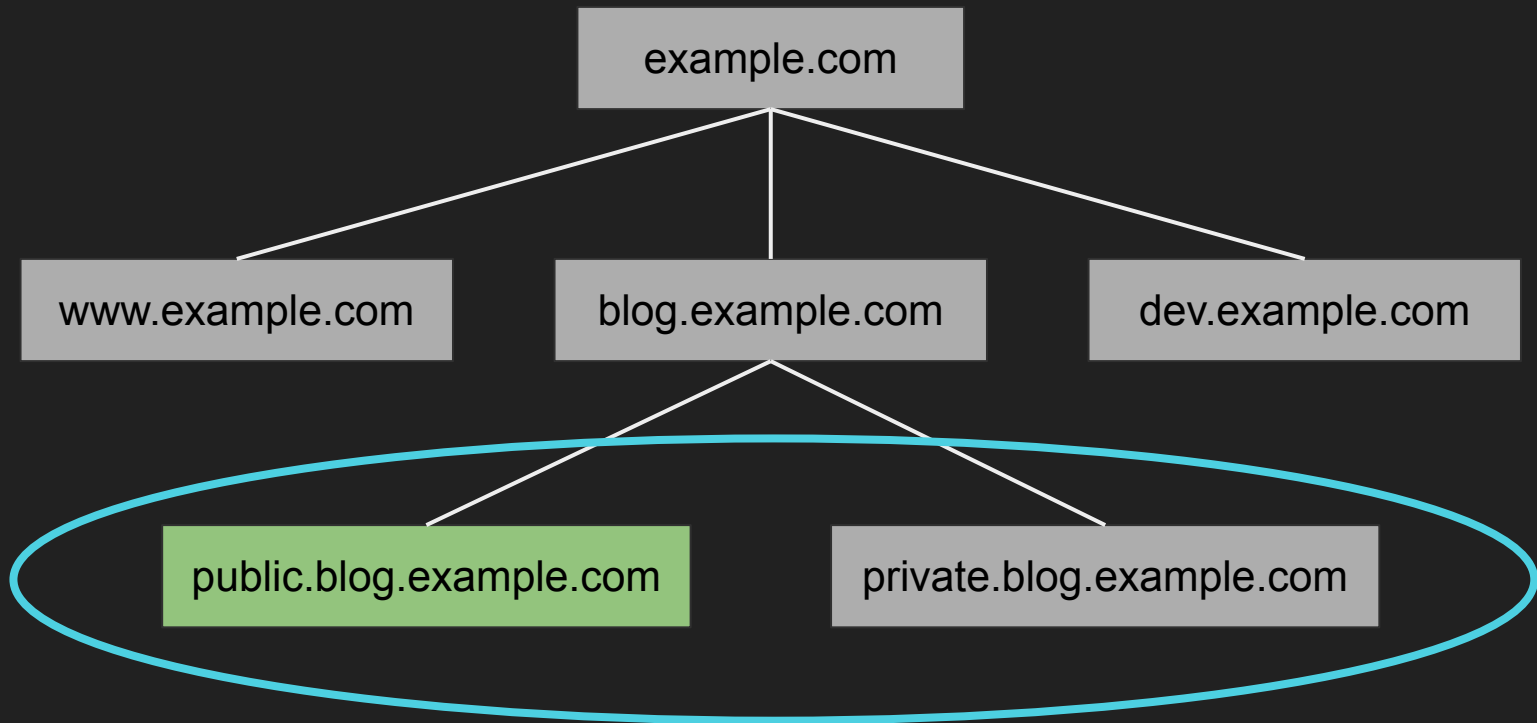
Same Origin Policy for Cookies

fetch	Scheme	Host	Port	CSP	N/A	Sec-Fetch-Mode
Cookies	Secure	Domain	N/A	Http-Only	Path	SameSite

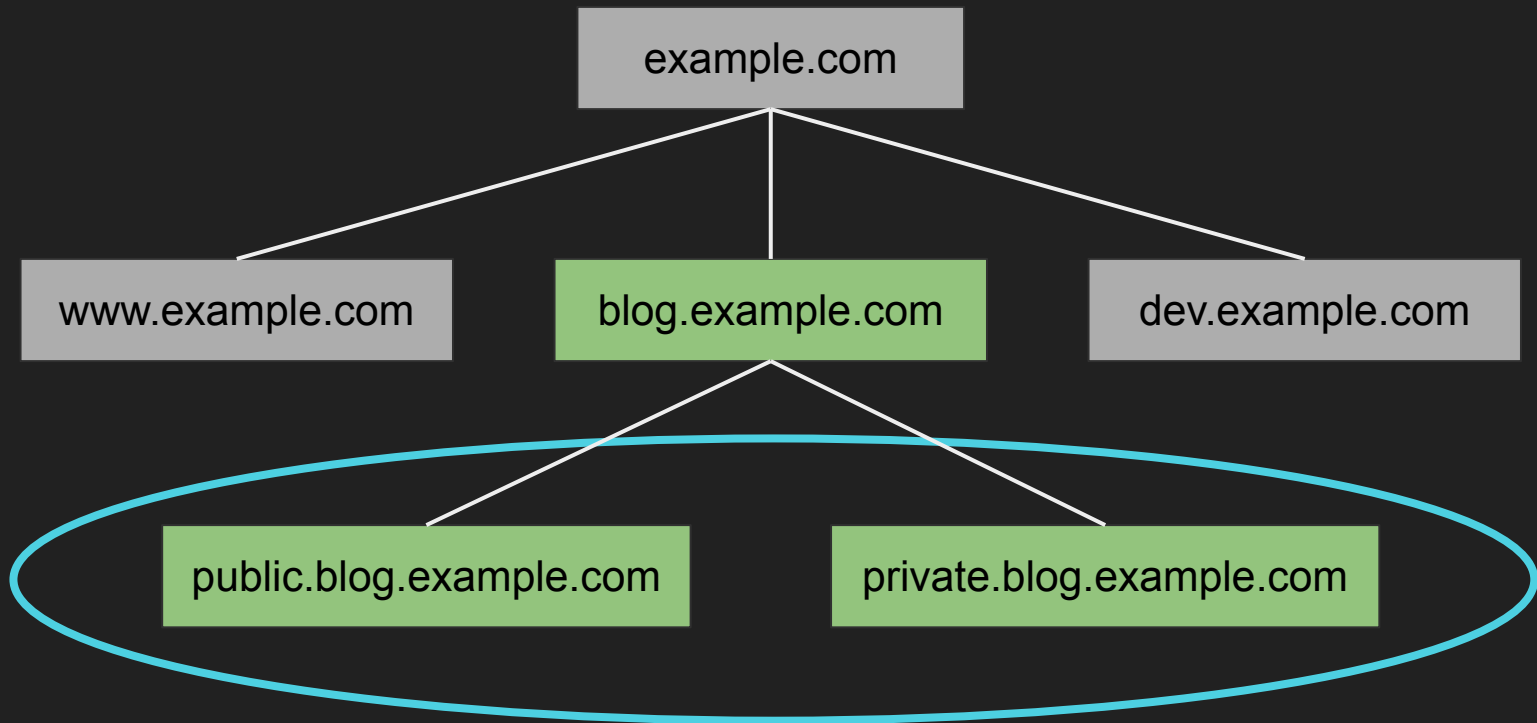
Cookies on Subdomains



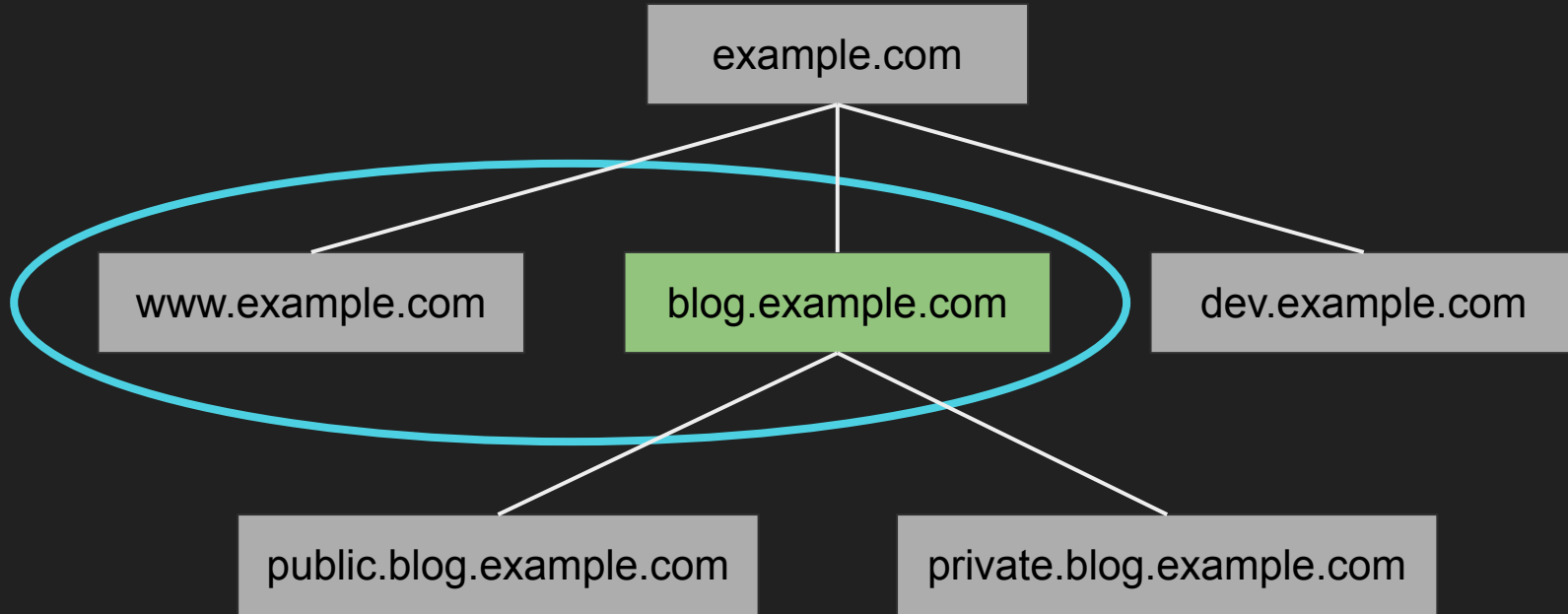
Cookies on Subdomains



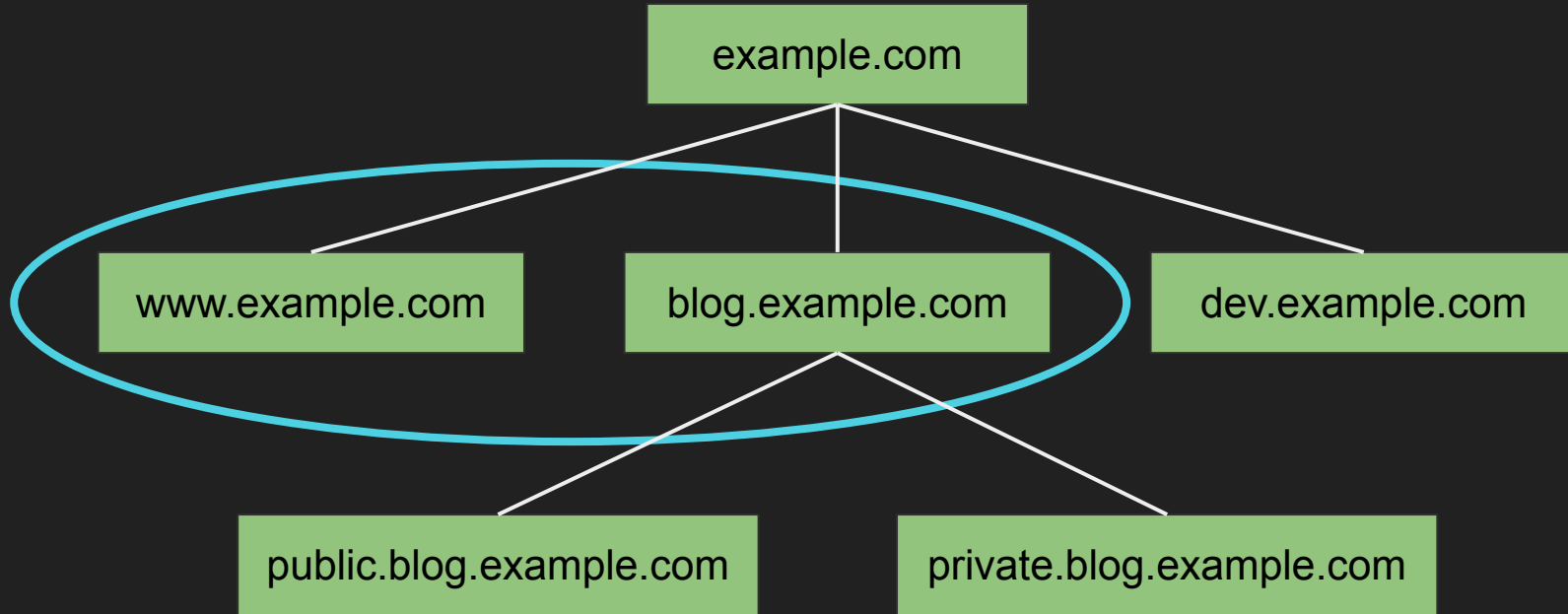
Set Cookie; Domain=blog.example.com



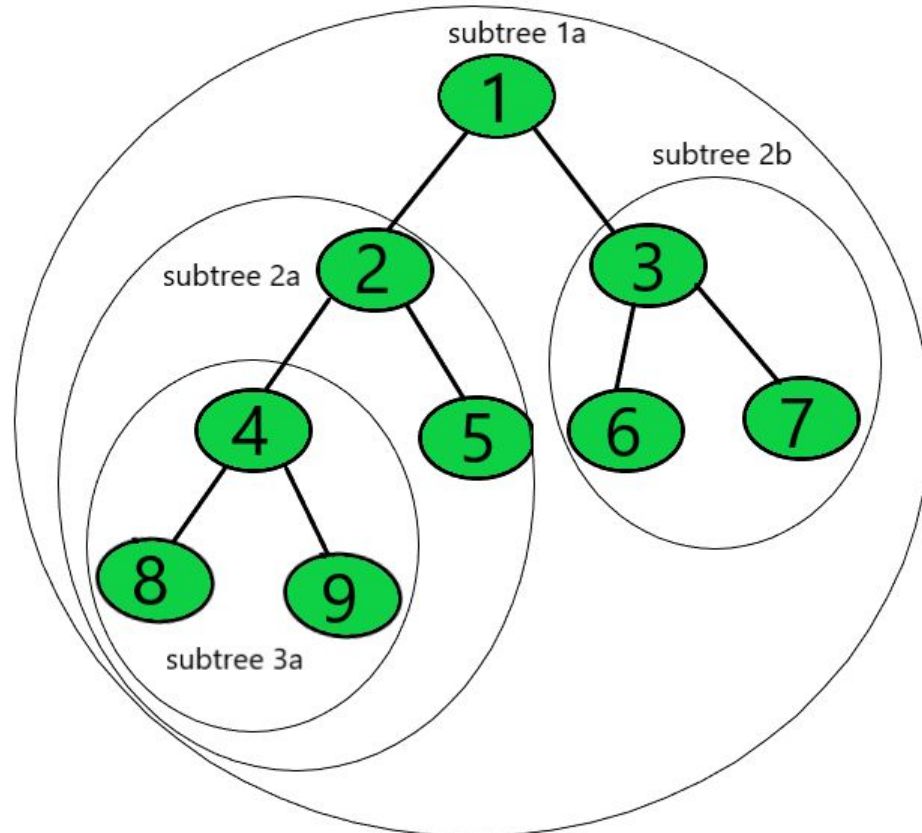
Cookies on Subdomains



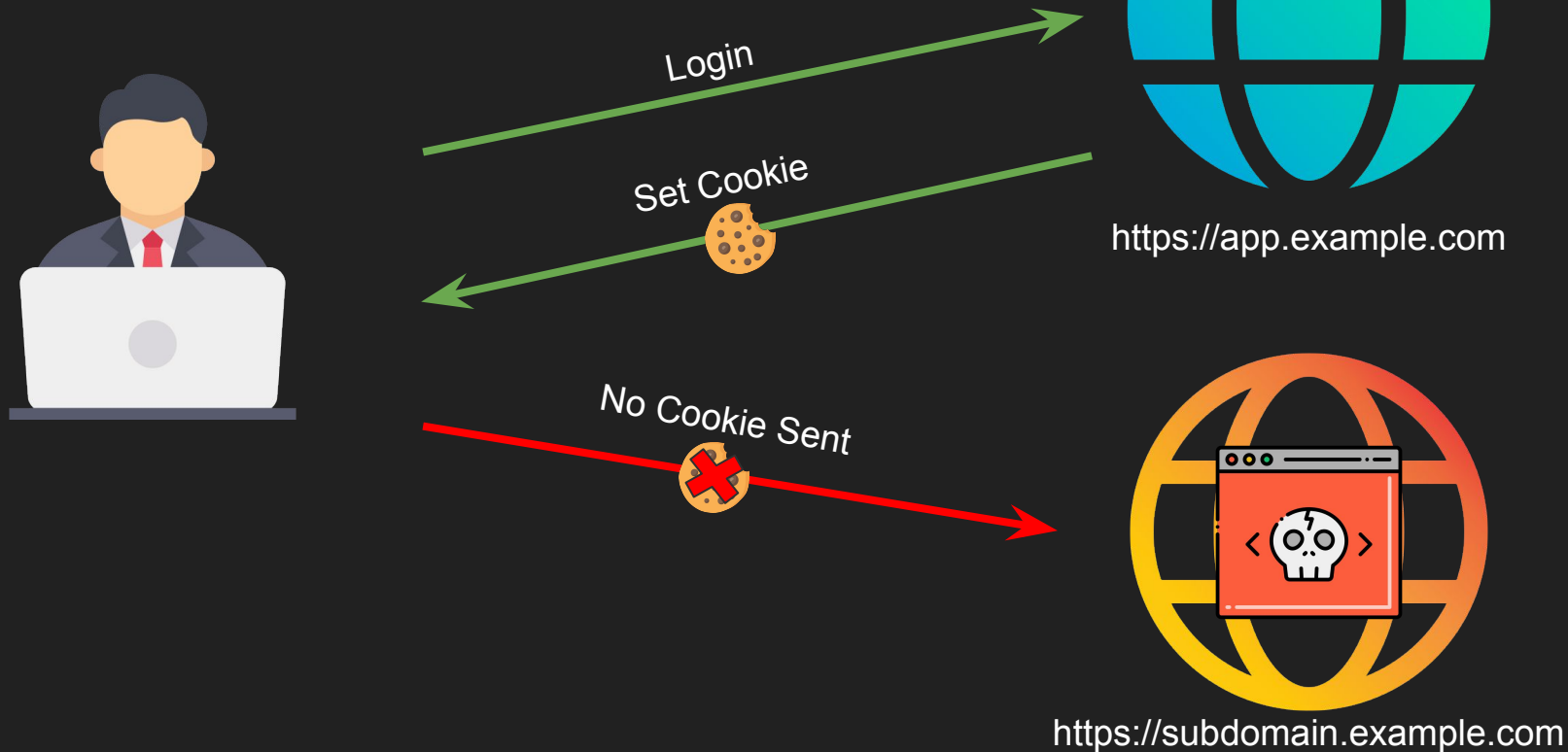
Set Cookie; Domain=example.com



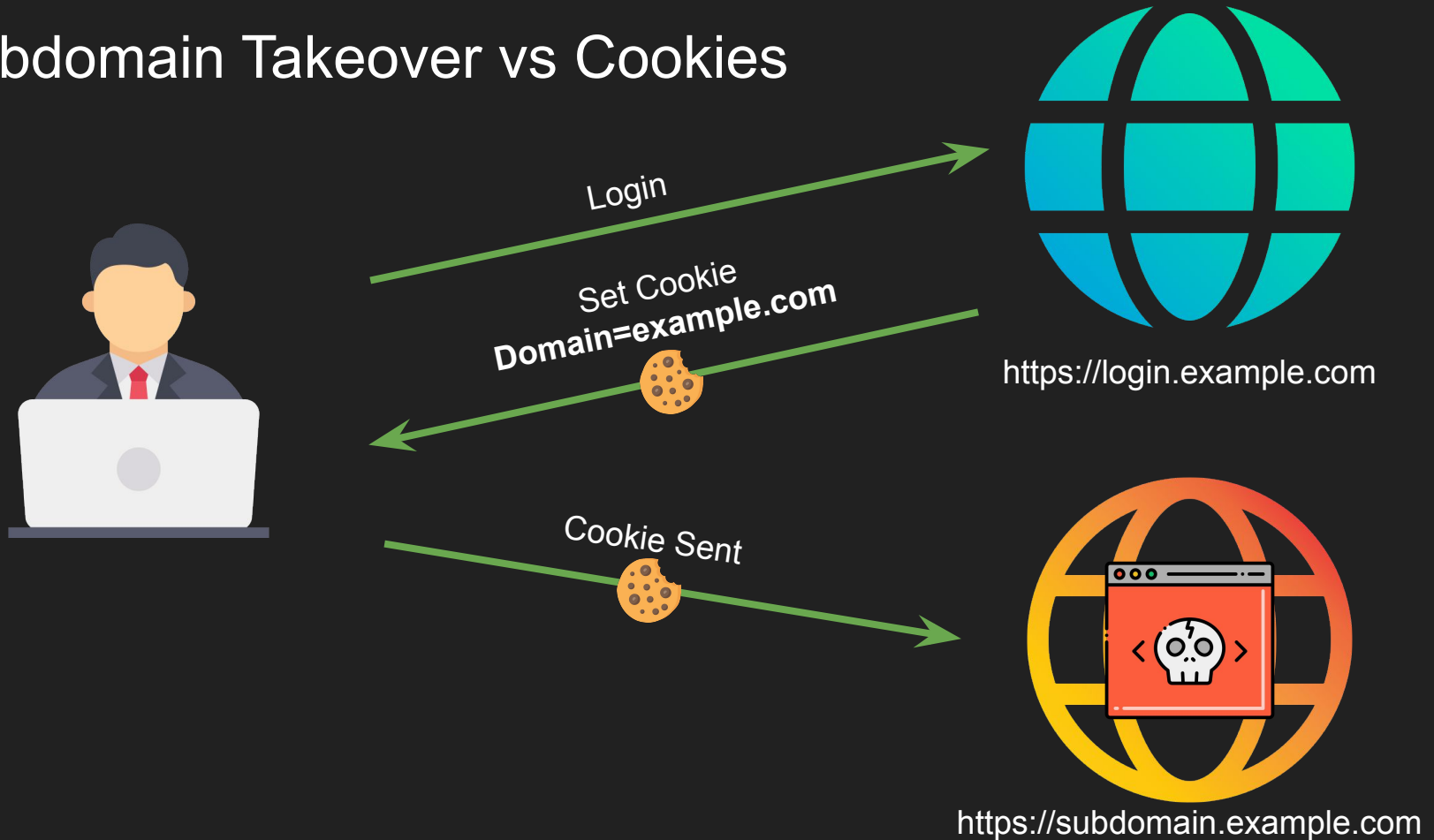
Subtrees



Subdomain Takeover vs Cookies



Subdomain Takeover vs Cookies



Subdomain Takeover vs Cookies



Browse / view ad

Set Cookie
Domain=example.com

Cookie Sent



<https://subdomain.example.com>



<https://billing.example.com>

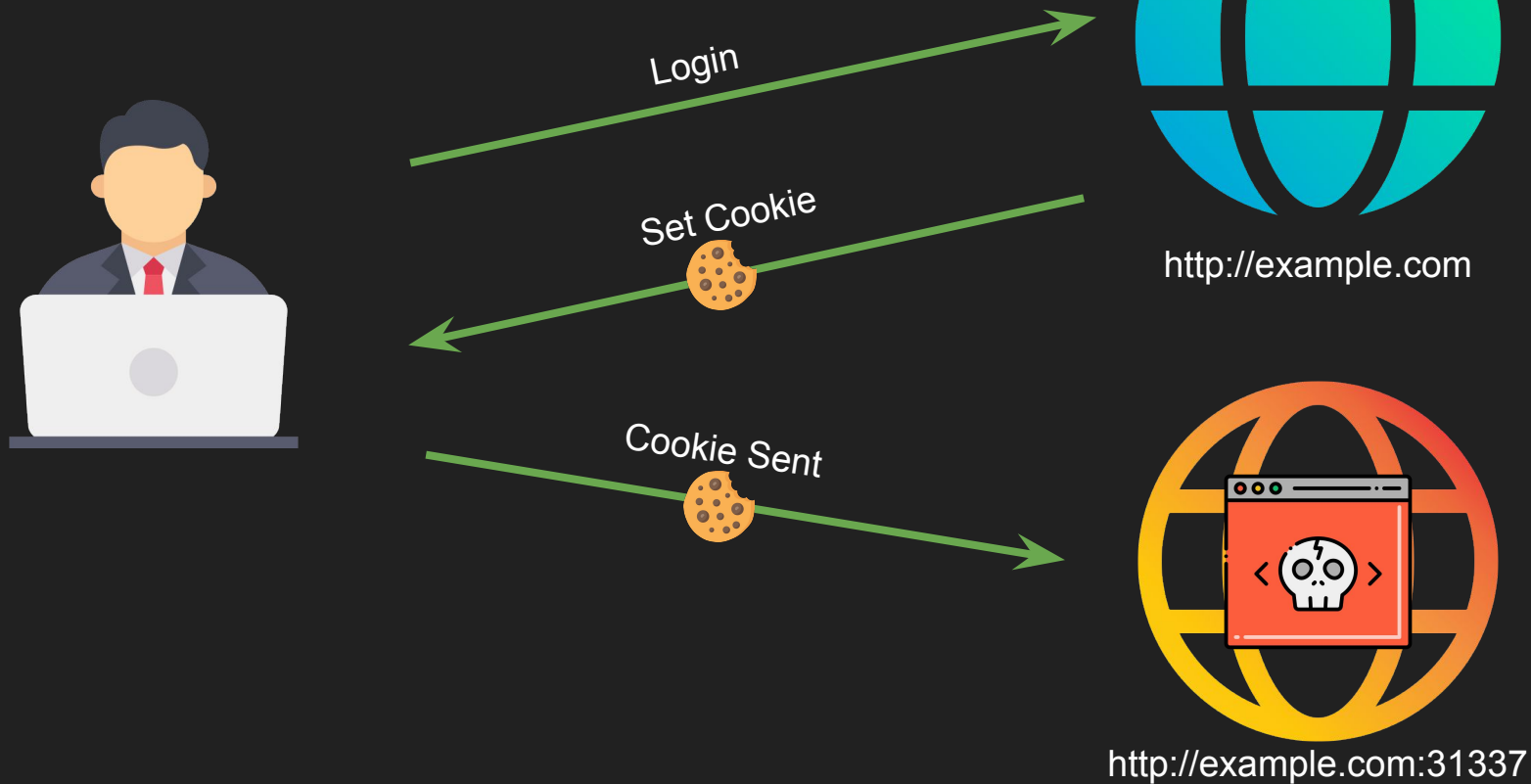


Domain locking: __Host-

Same Origin Policy for Cookies

XHR	Scheme	Host	Port	CSP	N/A	Sec-Fetch-Mode
Cookies	Secure	Domain	N/A	Http-Only	Path	SameSite

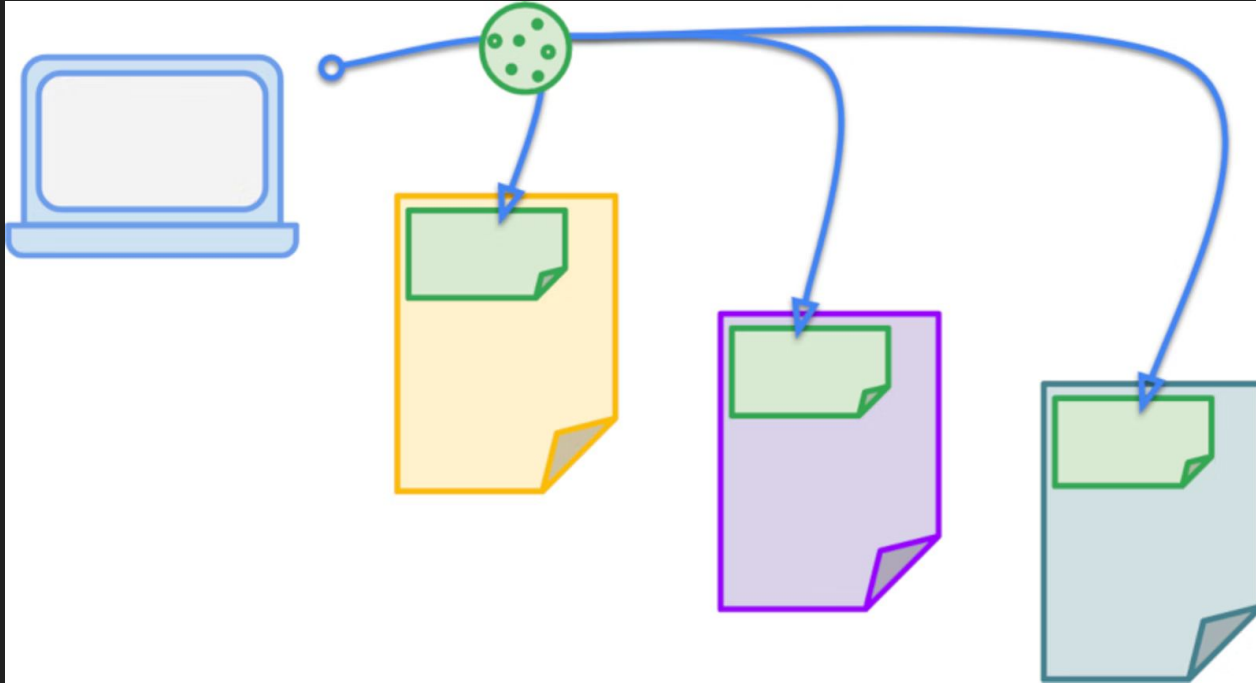
Cookies don't care about Ports



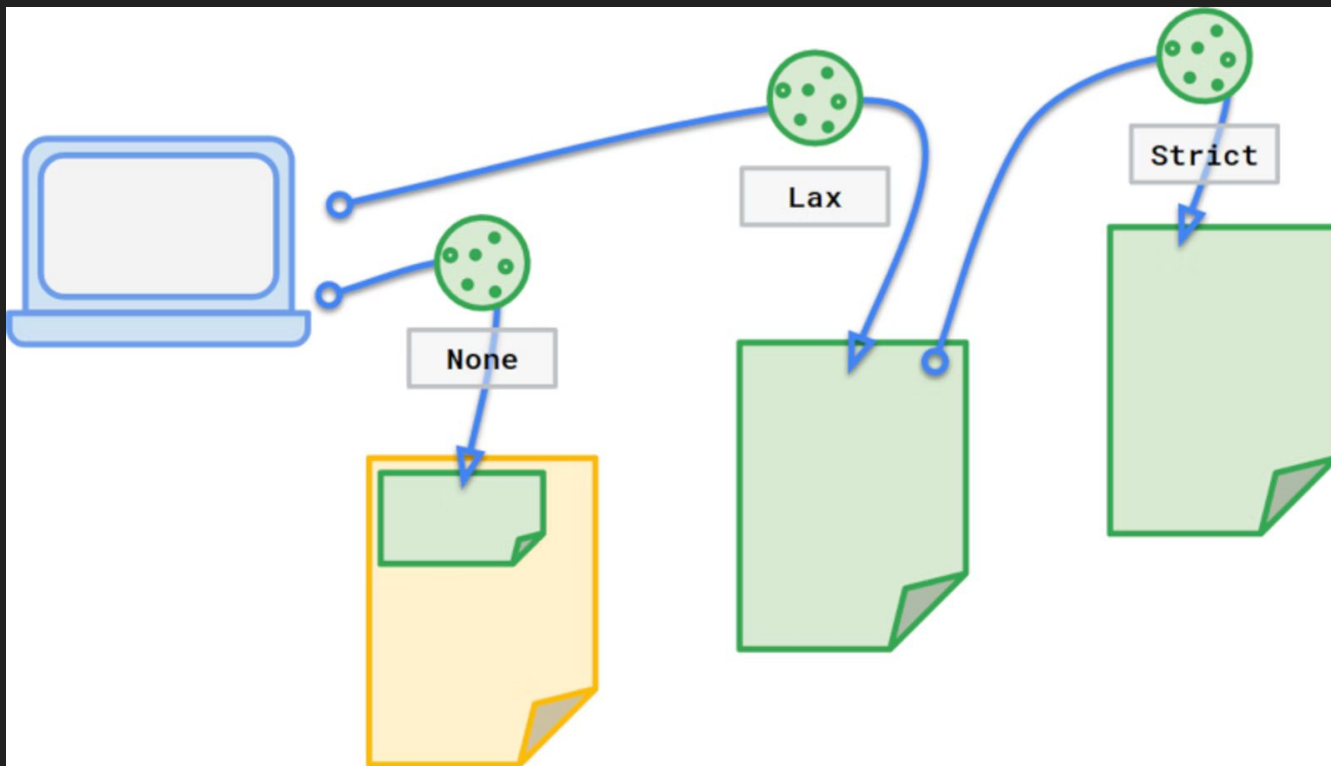
Same Origin Policy for Cookies

XHR	Scheme	Host	Port	CSP	N/A	Sec-Fetch-Mode
Cookies	Secure	Domain	N/A	Http-Only	Path	SameSite

Same-Site Cookies: 3rd party cookies



Same-Site Cookies: 3rd party cookies



CSRF: SameSite=None



Browse / view ad

"Make a POST request to
`https://example.com/change_email`"

POST
`https://example.com/change_email`
`email=attacker@evil.com`



`https://evil.com`



`https://example.com`

CSRF: SameSite=Lax or Strict



Browse / view ad

"Make a POST request to
`https://example.com/change_email`"



`https://evil.com`



POST
`https://example.com/change_email`
`email=attacker@evil.com`



`https://example.com`

CSRF: SameSite=Lax or Strict



Browse / view ad

"Make a POST request to
`https://example.com/change_email`"

POST
`https://example.com/change_email`
`email=attacker@evil.com`



`https://subdomain.example.com`



`https://example.com`

Browser Bugs (fixed!)

- [SameSite cookie bypass via redirect](#)
- [SameSite cookie bypass via link inside iframe](#)
- [SameSite cookie bypass via null principal](#) (Reader, sandbox iframes...)
 - [SameSite cookie Reader view patch bypass](#)
- [SameSite Cookie Bypass via BackgroundFetch](#)
- [SameSite policy bypassed with Service Worker FetchEvent](#)

Browser Quirks

- [SameSite cookie inconsistency via Drag & Drop](#)
- IE doesn't include ports in the Origin
- IE/Edge allows reading CSS rules cross-origin ([Same-Origin Policy: Evaluation in Modern Browsers](#))

Agenda

- whoami
- The Same-Origin Policy
- Cookies
- **DNS Rebinding and Demo**

DNS Rebinding

“The same-origin policy controls interactions between two different origins”

- [Mozilla Developer Network](#)

What a hostname resolves to is NOT part of the origin!

Origins:

- Protocol
- Host
- Port

`https://example.com:443`

DNS Resolution does not change the Origin

The browser window shows a tab titled 'rebind poc' with the address bar displaying 'rebind.nofe.cz:...' (highlighted with a red box). The page content includes the text 'rebind poc' and 'attacker website'.

The terminal window shows the command 'nslookup rebinder.nofe.cz' and its output:

```
C:\Users\cberman>nslookup rebinder.nofe.cz
Server: cdns01.comcast.net
Address: 2001:558:feed::1

Non-authoritative answer:
Name:     rebinder.nofe.cz
Address:  54.198.78.60
```

The 'Non-authoritative answer' section is highlighted with a red box.

The browser window shows a tab titled 'Directory listing for /' with the address bar displaying 'rebind.nofe.cz:...' (highlighted with a red box). The page content includes the text 'Directory listing for /' and a list of links: [2020autotax.pdf](#), [2021autotax.pdf](#), and [Ableton/](#).

The terminal window shows the command 'nslookup rebinder.nofe.cz' and its output:

```
C:\Users\cberman>nslookup rebinder.nofe.cz
Server: cdns01.comcast.net
Address: 2001:558:feed::1

Non-authoritative answer:
Name:     rebinder.nofe.cz
Address:  127.0.0.1
```

The 'Non-authoritative answer' section is highlighted with a red box.

DNS Rebinding Example

```
C:\WINDOWS\system32\cmd.exe

C:\Users\cberman>nslookup rebinder.nofe.cz
Server:  cdns01.comcast.net
Address:  2001:558:feed::1

Non-authoritative answer:
Name:     rebinder.nofe.cz
Address:  127.0.0.1

C:\Users\cberman>
```

rebind poc

rebinder.nofe.cz:8000/blank-poc.html

rebind poc

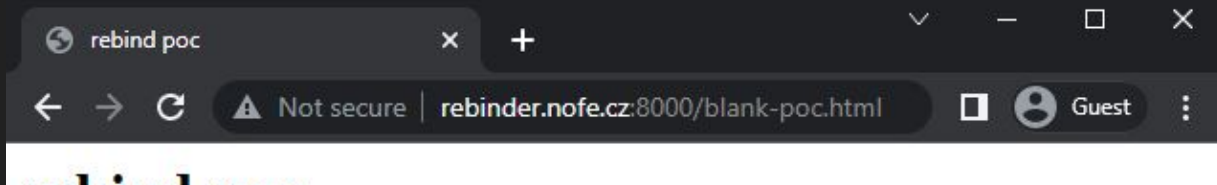
attacker website

```
>> fetch('http://rebinder.nofe.cz:8000')
    .then(response => response.text())
    .then(data => console.log(data.substring(270, 509)))

<> Promise { <state>: "pending" }
```

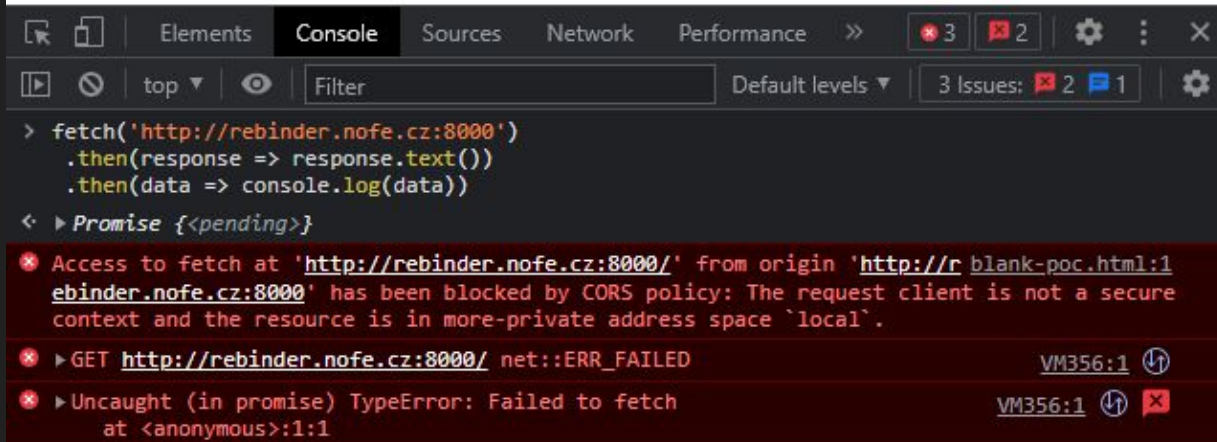
```
<li><a href="2020autotax.pdf">2020autotax.pdf</a></li>
<li><a href="2021autotax.pdf">2021autotax.pdf</a></li>
<li><a href="Ableton/">Ableton</a></li>
<li><a href="arch-backup/">arch-backup</a></li>
<li><a href="Arduino/">Arduino</a></li>
```

DNS Rebinding Counterexample - Chrome >94



Access to fetch at URL from origin URL has been blocked by CORS policy:

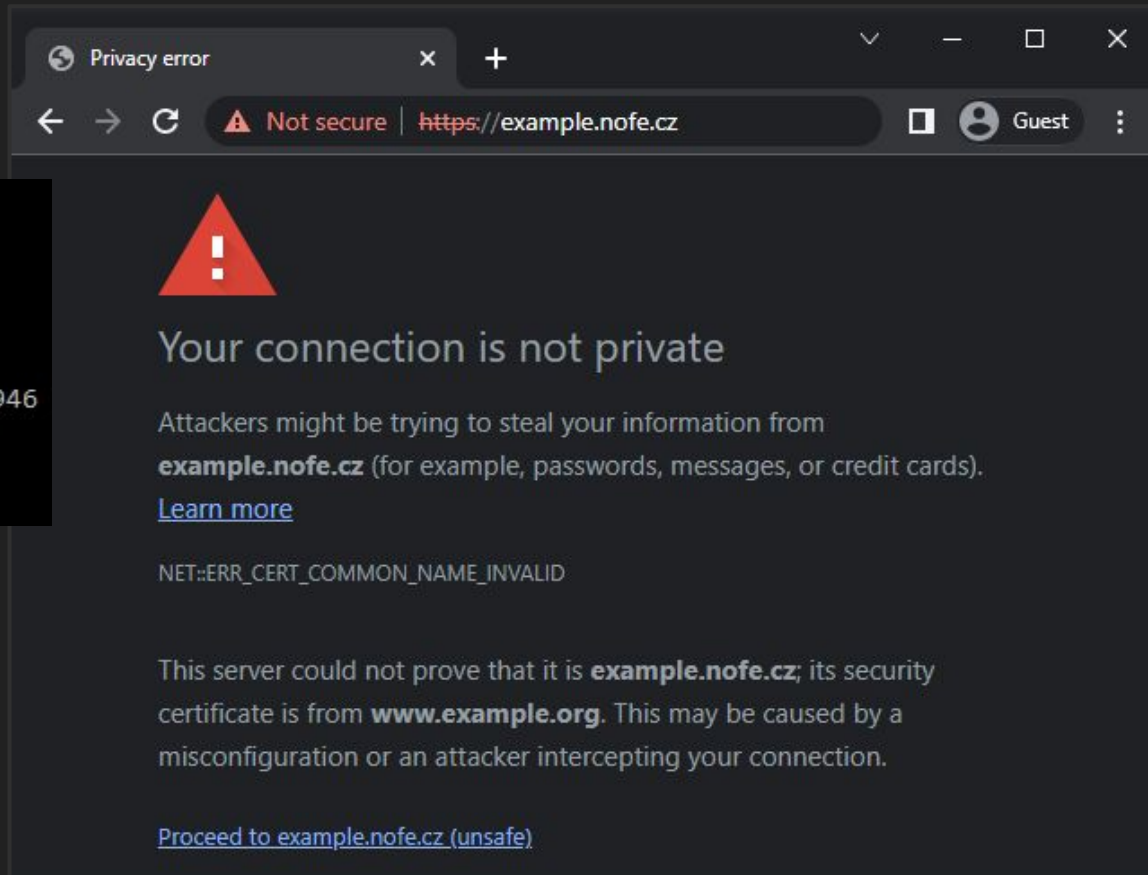
- The request client is not a secure context
- and the resource is in more-private address space `local`.



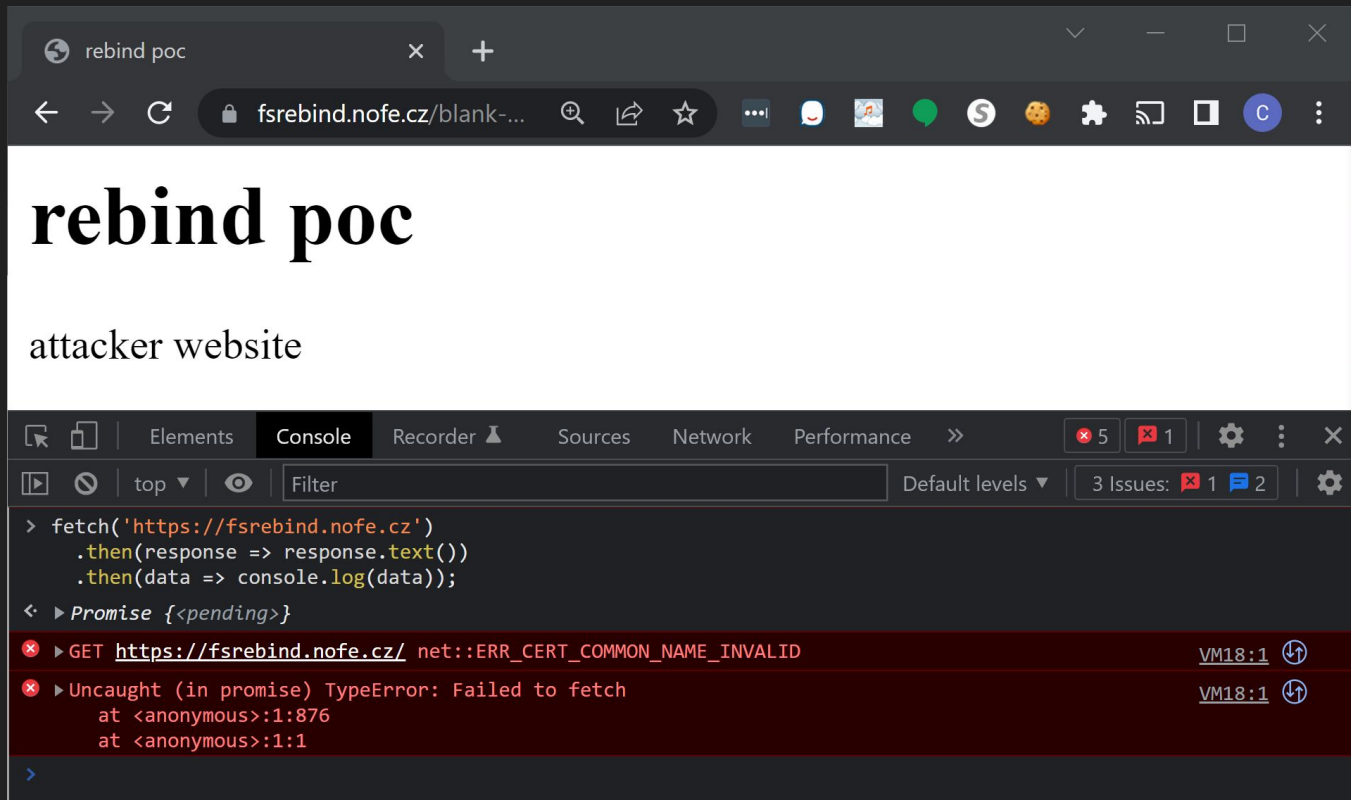
DNS Rebinding vs HTTPS

```
C:\Users\cberman>nslookup example.nofe.cz
Server:  cdns01.comcast.net
Address:  2001:558:feed::1

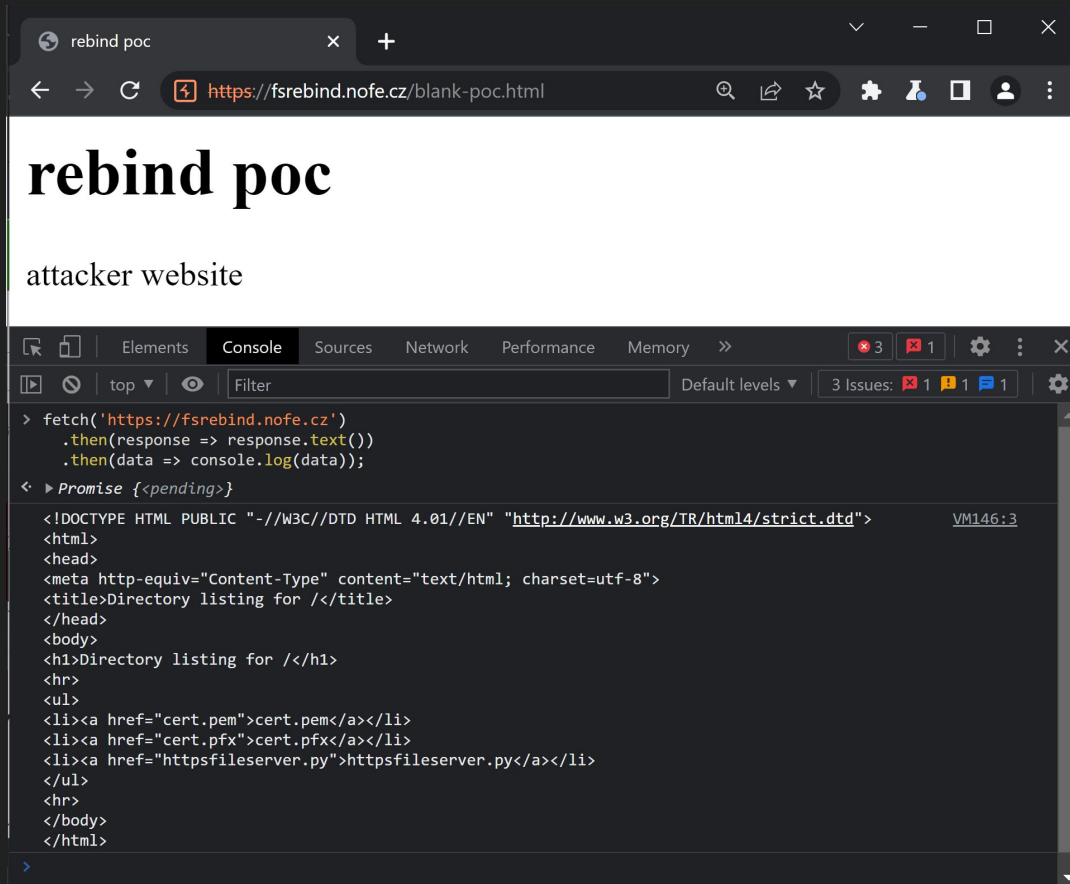
Non-authoritative answer:
Name:     example.com
Addresses: 2606:2800:220::1:248:1893:25c8:1946
          93.184.216.34
Aliases:  example.nofe.cz
```



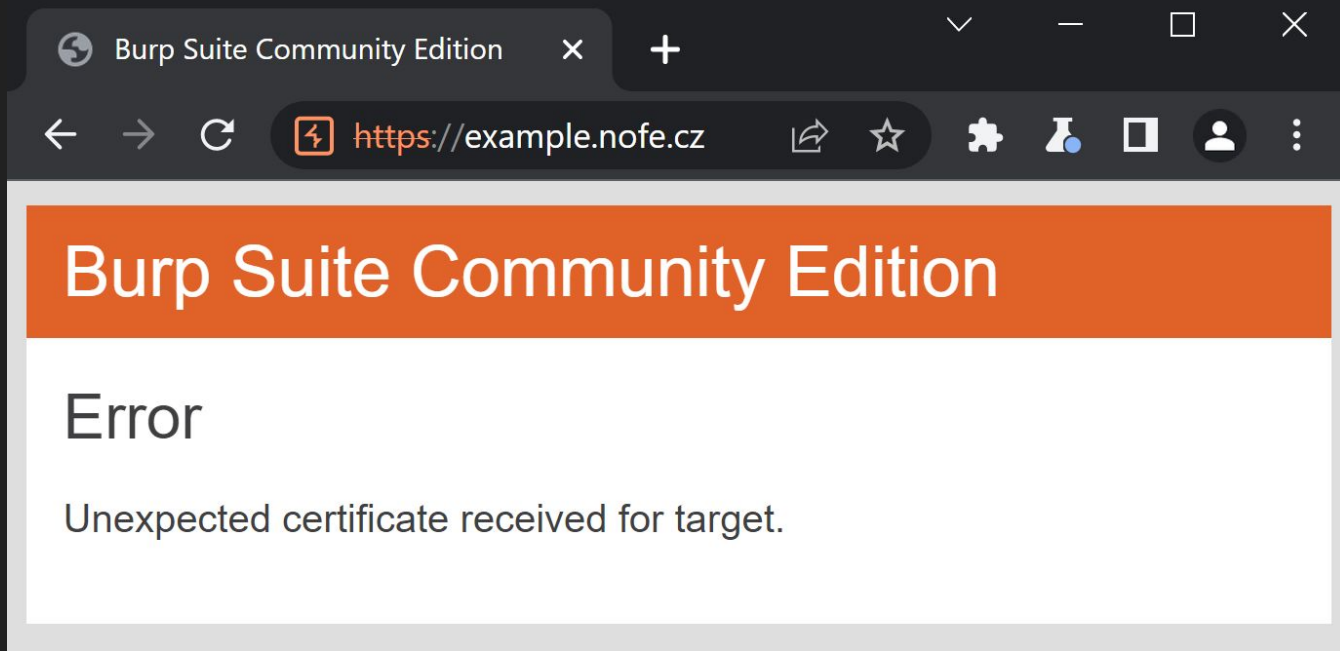
DNS Rebinding vs HTTPS



DNS Rebinding vs HTTPS: Without SSL Validation



Burp Suite v2022.5.1



Further topics

- Same-Origin Policy: Evaluation in Modern Browsers
 - Jörg Schwenk, Marcus Niemietz, and Christian Mainka
- Exploiting CORS Misconfigurations for Bitcoins and Bounties
 - James Kettle
- Cross-Site Leaks
 - Manuel Sousa, terjanq, Roberto Clapis, David Dworken, NDevTK, 1lastBr3ath, Brasco, rick.titor, Chris Fredrickson, jub0bs

Questions?

Citations

- [Private Network Access](#) (Block cross-site public to private address space requests)
- [Worker icons created by DinosoftLabs - Flaticon](#)
- [Web-site icons created by Ilham Fitrotul Hayat - Flaticon](#)
- [Hacker icons created by Talha Dogar - Flaticon](#)
- [Cookie icons created by juicy_fish - Flaticon](#)